
THE JOB NOBODY HANDS YOU A MANUAL FOR

The 5 things nobody tells you before you become a PM

Nobody hands you a manual for this job. You get a title, a calendar full of meetings, and a team that is quietly deciding whether you are worth listening to.

These are the five things I had to learn the slow way. None of them are theory. Every one of them is something I do.

1 Get your hands dirty

You do not need to write production code. You do need to understand what your team is actually building. Read the pull requests. Read the design documents and leave real feedback, not "looks good to me." Ask the question you think is too basic to ask, because half the room is wondering the same thing and nobody wants to go first.

EXAMPLE

Before your next design review, read the doc twice and write down two questions about tradeoffs the author did not address. Bring those instead of opinions about the outcome.

WHY THIS MATTERS: your engineers are deciding whether you are competent long before your manager is. They decide it from whether you understand the thing they are building. Technical credibility is not about writing code, it is about never being the person the team has to slow down for.

2 Learn to motivate and influence the team

You have no authority here. You cannot order anyone to do anything, so the only lever you have is whether people want to work with you.

Build confidence in the people around you and say so out loud. Reward good work in public, where it costs you nothing and is worth a great deal to them. A kudos goes further than you think, especially from someone with no obligation to give one.

EXAMPLE

When someone on your team does something good, tell their manager in writing. Copy them. It takes ninety seconds and it is the single highest return thing you can do in a week.

WHY THIS MATTERS: influence compounds. The team that trusts you brings you problems early, while they are still small. The team that does not brings them to you at the end, when they are expensive.

3 Make your engineers' job easier

Engineers focus on the problem directly in front of them. That is the job and it is the right instinct. Your job is everything else.

Clear the path. Chase the external dependency before it becomes a blocker. Find out that the other team's API is slipping while there is still time to plan around it, not the morning it breaks your sprint.

EXAMPLE

Once a week, list every external thing your team is depending on and ask yourself which one you have not confirmed in the last ten days. Go confirm it.

WHY THIS MATTERS: you cannot promise your team a steady stream of perfectly scoped work. What you can promise is that you will never waste their time. Say that out loud to them, then hold to it. It changes how a team treats you.

4 Understand the value and the money behind your product

Know what your product is worth and why anyone pays for it. Then make sure your engineers know it too.

Most teams are handed the what and never the why. When you connect the work to the money, people make better decisions without asking you, because they finally have enough context to judge a tradeoff on their own.

EXAMPLE

In your next planning session, put a number next to each item. Revenue at stake, cost avoided, customers affected. If you cannot find a number for something, that is worth knowing before you build it.

WHY THIS MATTERS: the why is the best driver you have. It also protects you. When priorities shift, the person who can explain the business case keeps their roadmap. The person who only has a list of features loses it.

5 Don't lie to leadership to sound better

You are going to miss features and timelines. Everyone does. The temptation is to smooth it over, bury it in a status update, or find a way to make a slip sound like a plan.

Do not do that. Tell them what happened, what you learned, and what you are doing about it. Tell a story with a real arc instead of a headline with no substance underneath it.

EXAMPLE

Instead of "the feature slipped from the fifteenth to the twenty-ninth due to bugs," try: the team found critical issues during testing, we fixed them before they reached customers, and the tests we added will catch this class of bug earlier next time. Same facts. One version builds confidence.

WHY THIS MATTERS: you will not be remembered for the quarter you missed. You will be remembered for how you handled it. Trust is built in public when things go wrong, and it is spent quietly when you shade the truth and someone notices.

None of this is complicated. It is just the part nobody tells you before you take the job.

You will mess up. What matters is how you recover, and whether the people around you believe you when you tell them what happened.